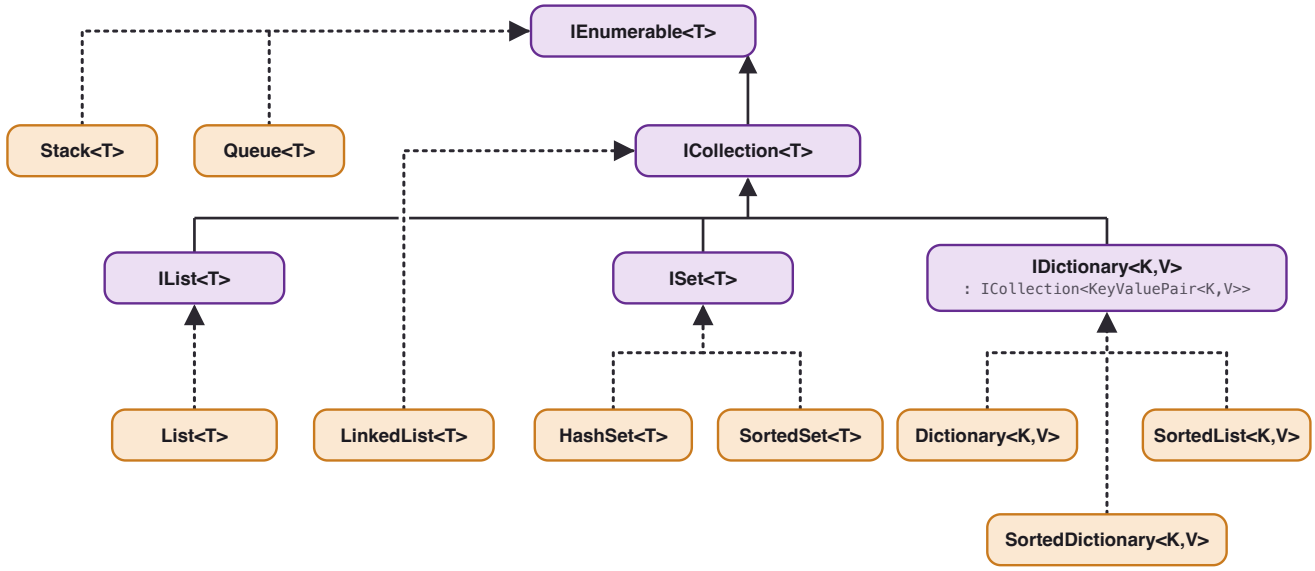
Colecțiile din .NET — Schema

Cine pe cine moștenește, în `System.Collections.Generic` · aceeași ierarhie în .NET Framework 4.x, la olimpiadă

Interfață · Clasă · → extinde (interfață din interfață) · ⇢ implementează

1. Cine pe cine moștenește

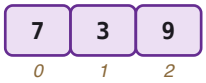


2. Ce colecție alegi

Întrebarea ta	Colecția	Ce folosești
„Am o înșiruire în ordine, cu poziții”	`List<T>`	`Add`, `Insert`, `RemoveAt`, `lista[i]`, `Sort`
„Adaug și scot des la capete, nu-mi trebuie poziții”	`LinkedList<T>`	`AddFirst`, `AddLast`, `RemoveFirst`, `RemoveLast`
„Caut ceva după o cheie (nume, cod)”	`Dictionary<K, V>`	`TryGetValue`, `ContainsKey`, `d[cheie] = valoare`
„Caut după cheie și vreau cheile în ordine”	`SortedDictionary<K, V>`	ca la `Dictionary`; `foreach` dă cheile crescător; cheile comparabile
„Cheile în ordine, dar și a i-a cheie”	`SortedList<K, V>`	ca la `Dictionary` + `Keys[i]`, `Values[i]`
„Am văzut deja asta?” / „fără dubluri”	`HashSet<T>`	`Add` (întoarce `false` dacă exista), `Contains`, `UnionWith`
„Fără dubluri și în ordine”	`SortedSet<T>`	`Add`, `Contains`, `Min`, `Max`; `foreach` crescător; elementele comparabile
„Procesez în ordinea sosirii”	`Queue<T>`	`Enqueue`, `Dequeue`, `Peek`
„Mă întorc la ultimul lucru”	`Stack<T>`	`Push`, `Pop`, `Peek`
„Știu sigur câte sunt, nu se schimbă”	tablou `T[]`	`t[i]`, `Length`, `Array.Sort`

T[]

Mărime fixă, dată la creare; poziții de la 0.



Length = 3
t[0] .. t[2]

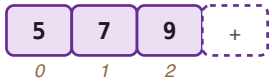
`t[i]`	citește / scrie
`Length`	câte are
`Array.Sort(t)`	sortează pe loc
`Array.IndexOf(t, x)`	poziția sau -1

⚠ `t[t.Length]` → IndexOutOfRangeException.

↔ la tine: tabloul din `ListaTablou`

List<T>

Tablou care crește singur; are poziții.



Add(x) pune aici
Count = 3

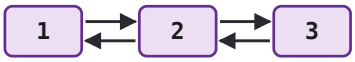
`Add(x)`	la sfârșit
`Insert(i, x)`	pe poziția i
`RemoveAt(i)`	scoate de pe i
`Remove(x)`	prima apariție → bool

⚠ `Add` / `Remove` în `foreach` pe ea → InvalidOperationException.

↔ la tine: `ListaTablou`

LinkedList<T>

Noduri legate; rapidă la capete, fără poziții.



`AddFirst(x)`	la început
`AddLast(x)`	la sfârșit
`RemoveFirst()`	scoate primul
`RemoveLast()`	scoate ultimul

⚠ `l.First` e un nod: valoarea e `l.First.Value`. Fără `l[i]`.

↔ la tine: `ListaInlantuita`

Dictionary<K, V>

Perechi cheie → valoare; cheile sunt unice.



d["ana"] → 18
cheia o singură dată

`d[k] = v`	adaugă sau înlocuiește
`Add(k, v)`	doar cheie nouă
`ContainsKey(k)`	există cheia?
`TryGetValue(k, out v)`	caută o dată

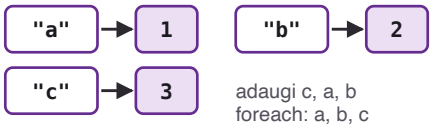
`if (d.TryGetValue("ana", out int v))`
out = întoarce **bool** și pune valoarea în `v` dacă există (altfel `v` = 0 / null).

⚠ `d[k]` pe cheie lipsă → KeyNotFoundException; folosește `TryGetValue`.

↔ la tine: `DictionarHash` (`TryToTake` ↔ `TryGetValue`, dar acesta nu scoate cheia)

SortedDictionary<K, V> + SortedList<K, V>

Dicționar cu cheile mereu crescătoare.



adaugi c, a, b
foreach: a, b, c

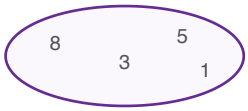
`d[k] = v, Add, TryGetValue`	ca la Dictionary
`foreach`	cheile crescător
`Keys[i], Values[i]`	a i-a cheie (SortedList)
`IndexOfKey(k)`	poziția cheii (SortedList)

⚠ `Keys[i]` există doar la SortedList; la SortedDictionary nu compilează.

Cheile trebuie comparabile: vezi `IComparable` / `IComparer`

HashSet<T>

Mulțime: fiecare valoare o dată, fără ordine.



Add(5) → true
Add(5) → false
fără ordine

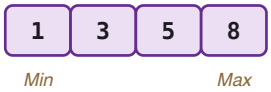
`Add(x)`	false dacă exista
`Contains(x)`	direct, rapid
`Remove(x)`	→ bool
`UnionWith(o)`	reuniune, pe loc

⚠ `UnionWith` / `IntersectWith` modifică mulțimea ta, nu întorc una nouă.

↔ la tine: `MultimeHash`

SortedSet<T>

Mulțime fără dubluri, mereu în ordine.



crescător, fără dubluri

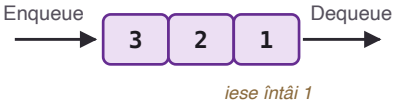
`Add(x)`	false dacă exista
`Contains(x)`	există?
`Min / Max`	primul / ultimul
`Remove(x)`	→ bool

⚠ Fără `s[i]` (nu compilează); primul element e `Min`.

Elementele trebuie comparabile: vezi `IComparable` / `IComparer`

Queue<T>

Primul intrat, primul ieșit (FIFO).



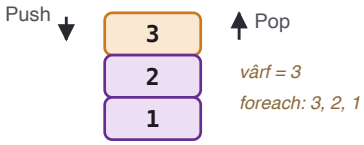
`Enqueue(x)`	la coadă
`Dequeue()`	scoate din față
`Peek()`	se uită, nu scoate
`Count`	câte așteaptă

⚠ `Dequeue` / `Peek` pe coadă goală → InvalidOperationException; verifici `Count > 0`.

↔ la tine: `CoadaCirculara`

Stack<T>

Ultimul intrat, primul ieșit (LIFO).



`Push(x)`	pune în vârf
`Pop()`	scoate din vârf
`Peek()`	se uită, nu scoate
`Count`	câte are

⚠ `foreach` pornește de la vârf (3, 2, 1), nu de la primul pus.

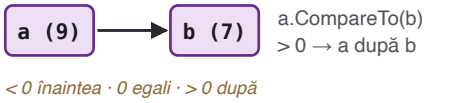
↔ la tine: `StivaTablou`

La olimpiadă (.NET Framework 4.x) NU există — compilat cu 4.7.2

Nu există	Scrii așa
`Dictionary.TryAdd(k, v)`	`if (!d.ContainsKey(k)) { d.Add(k, v); }`
`Stack.TryPop / TryPeek`	`if (s.Count > 0) { x = s.Pop(); }`
`Queue.TryDequeue / TryPeek`	`if (q.Count > 0) { x = q.Dequeue(); }`
`EnsureCapacity(n)`	nu o folosești
`HashCode.Combine(a, b)`	`a.GetHashCode() * 31 + b.GetHashCode()`

IComparable<T>

Ordinea naturală: clasa își știe singură locul.



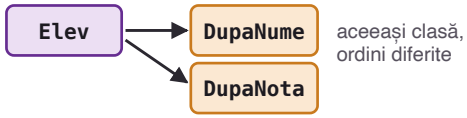
`int CompareTo(T altul)`	o scrii în clasă
`< 0`	this înaintea lui altul
`= 0`	egali ca ordine
`> 0`	this după altul

```
class Elev : IComparable<Elev>
{ public double Nota;
  public int CompareTo(Elev altul)
  { return Nota.CompareTo(altul.Nota); } }
```

O folosesc singure: `lista.Sort()`, `Array.Sort(t)`, `SortedSet`, `SortedDictionary`, `SortedList`, `Min` / `Max`

IComparer<T>

O ordine din afară: clasă proprie, `Elev` rămâne neatins.



`int Compare(T a, T b)`	aceeași regulă ca CompareTo
`lista.Sort(c)`	List
`new SortedSet<T>(c)`	și SortedDictionary / SortedList
`Comparer<T>.Default`	comparatorul implicit (CompareTo)

```
class DupaNume : IComparer<Elev>
{ public int Compare(Elev a, Elev b)
  { return string.Compare(a.Nume, b.Nume,
      StringComparison.Ordinal); } }
```

`new SortedSet<Elev>(new DupaNume())` — fără `IComparable` pe `Elev`

IComparable vs IComparer

	`IComparable<T>`	`IComparer<T>`
Unde se scrie	în clasa elementului	într-o clasă separată
Metoda	`CompareTo(altul)`	`Compare(a, b)`
Câte ordini	una singură	oricâte (o clasă per ordine)
Se modifică clasa?	da	nu
Cum ajunge la colecție	singură, nu o dai nicăieri	o dai: `Sort(c)` / constructor
Când alegi	ordinea evidentă a clasei	altă ordine sau mai multe ordini
`HashSet` / `Dictionary`	nu folosesc `CompareTo`: merg pe `Equals` + `GetHashCode` (sau `IEqualityComparer<T>` în constructor)	

⚠ **1.** Element necomparabil (fără `IComparable`, fără comparer): primul `Add` trece, al doilea → `SortedSet` / `SortedDictionary`: ArgumentException „At least one object must implement IComparable.”; `SortedList`, `lista.Sort()`, `Array.Sort`: InvalidOperationException „Failed to compare two elements in the array.”, cu InnerException = ArgumentException (același mesaj). Și `new SortedSet<T>(colecție)` cu 2+ elemente: la fel ca `SortedList`.

⚠ **2.** În `SortedSet` / `SortedDictionary`, `CompareTo == 0` înseamnă același element. Ana și Bob, ambii cu nota 9 (`Equals` = false): `SortedSet.Add` → true, apoi false, `Count` = 1. `SortedDictionary.Add` → ArgumentException „An item with the same key has already been added.” `HashSet` (pe `Equals`): true, true, `Count` = 2. Vrei ambii: la nota egală, compară și după nume.